

Dilithium for Memory Constrained Devices 논문 리뷰

<https://youtu.be/wqlfyB772V0>

Dilithium for Memory Constrained Devices

- 해당 논문(AfricaCrypt 2022)이 pqm4 에 공개된 시점 기준 Dilithium의 clean 코드와 opt 구현의 메모리 사용량 : 50~100 KiB
- **Pure C 구현(하드웨어 가속기 및 ASM 사용 X)**
- **연구 목표 (Target Constraints)**
 - 타겟: Cortex-M0+ 등 초소형 IoT 장치
 - 메모리 제약: 8 KiB 이하의 SRAM 환경에서 동작하도록 최적화
 - 실험 및 검증 환경 (Test Platform)
 - 사용 보드: STM32F4 Discovery (STM32F407)
 - 선정 이유: pqm4 벤치마킹 프레임워크 호환성 및 스택 사용량 측정을 위함
- 성능(각 작업이 실행될 때의 필요한 Peak Memory)
 - 키 생성 및 서명 생성 : 각각 7KiB 미만
 - 서명 검증 : 3KiB 미만
 - 총합이 10KiB 넘을거 같은데 여기 측정된 수치는 동시에 필요한 메모리 양이 아님
 - 작업이 독립적으로 일어나 순차적으로 연산 후, 메모리도 바로 비우기 때문



Dilithium for Memory Constrained Devices

- 딜리시움 단점

- 비교적 큰 키와 서명 크기
- 중간 데이터 저장을 위한 스택 공간을 과도하게 사용함
따라서, 메모리 사용이 많음

- 사용 기법

- **특정 시점에 다른 값이 사용했던 메모리 공간을 재사용**
 - 생명주기 : 변수의 수명으로 변수가 초기화된 시점부터 마지막으로 사용될 때까지의 시간
 - 생명 주기가 끝나면? 변수가 마지막으로 사용된 후에는 그냥 죽은 상태
 - 할당? 변수 저장을 위해 특정 메모리 예약
 - 따라서, 2개의 변수의 수명이 겹치는 것은 알고리즘의 특정 시점에 2개의 변수가 살아있는 상태
 - 이 경우 수명이 겹치는 2개의 변수는 동일한 메모리 위치 공유 못함

기법 1) Streaming A and y

- 적용 전) Dilithium 2 : 49KiB / Dilithium 3: 80 KiB / Dilithum 5 : 116 KiB

- **기법 1) Streaming A and y(On the fly)**

- 서명 생성 알고리즘에 행렬 A가 메모리 차지 제일 많이 함(행렬 A : $k \times l$ 구조, $k * l$ KiB)

보안 레벨 (NIST Level)	k (행의 개수)	l (열의 개수)	행렬 A의 원소 수 ($k \times l$)
Dilithium-2	4	4	16개
Dilithium-3	6	5	30개
Dilithium-5	8	7	56개

- 행렬 A의 각 원소는 $n=256$ 개의 계수를 가진 다항식
 - 기존 구현에서는 각 계수를 4Byte(32비트) 정수로 저장하기 때문에 다항식 하나는 1024byte(=1KiB)를 차지
- 적용 후) Dilithium 2 : 20KiB / Dilithium 3: 35 KiB / Dilithum 5 : 63 KiB 정도 각각 절감
ex) $49-20= 29$ KiB 이렇게 메모리 사용량 감소

w 벡터 압축 원리

- 딜리시움 다항식 구조 : n의 256개의 계수(Coefficient)로 구성
 - $f(x) = a_0x^0 + a_1x^1 + \dots + a_{255}x^{255}$
- 값의 범위 (q) : 모든 계수는 소수 $q = 8,380,417$ 보다 작음
- 일반적인 구현(clean)은 32-bit(4-Byte) 정수형 사용
 - 하지만 실제 데이터는 23-bit면 충분하기 때문에 9bit 낭비
- 24-bit Packing
 - 8-bit 낭비를 제거하고 24-bit로 압축
 - 23 bit가 아닌 24 bit 인 이유는 8의 배수(1-Byte)가 정렬 효율성을 위해 선택

기법 2) w 벡터 압축

- w? 서명 생성 과정에서 가장 먼저 계산되는 핵심 중간값

- w = Ay

- A: 공개된 행렬(k x l 행렬)
- y: 서명마다 새롭게 뽑는 랜덤벡터(마스킹 벡터)
- W는 k개의 다항식으로 이루어짐 / 원소가 k개, 각 원소는 256개의 계수를 갖는 다항식

- W의 압축 저장이 가능한 이유?

- w의 각 계수는 32비트 정수이지만 24비트로 압축해서 저장 가능
- Why? $Q=8,380,417(2^{23})$ 안에서 연산되기 때문

- 각 계수를 q로 나눈 나머지 구하기
- 이 값을 24비트에 딱 채워서 패킹
- 다항식 하나당 1024byte(32bit*256) 필요했던 것이 **768-Byte(24bit*256)으로 줄어 들음**

- 이 과정에서 메모리는 아끼지만 연산 오버헤드 생김

- 챌린지 c 생성 이후에도 w 원본이 필요하여 메모리 해제 불가(Lifetime Overlap)

Algorithm 2 Dilithium signature generation (taken from [18])

Input: Secret key sk and a message M .

Output: Signature $\sigma = \text{Sign}(sk, M)$.

```

1:  $\mathbf{A} \in R_q^{k \times \ell} := \text{ExpandA}(\rho)$  ▷  $\mathbf{A}$  is generated in NTT domain as  $\hat{\mathbf{A}}$ 
2:  $\mu \in \{0, 1\}^{512} := H(tr \parallel M)$ 
3:  $\kappa := 0, (\mathbf{z}, \mathbf{h}) := \perp$ 
4:  $\rho' \in \{0, 1\}^{512} := H(K \parallel \mu)$  (or  $\rho' \leftarrow \{0, 1\}^{512}$  for randomized signing)
5: while  $(\mathbf{z}, \mathbf{h}) = \perp$  do ▷ Pre-compute  $\hat{\mathbf{s}}_1 := \text{NTT}(\mathbf{s}_1)$ ,  $\hat{\mathbf{s}}_2 := \text{NTT}(\mathbf{s}_2)$ , and  $\hat{\mathbf{t}}_0 := \text{NTT}(\mathbf{t}_0)$ 
6:    $\mathbf{y} \in S_{\gamma_1}^\ell := \text{ExpandMask}(\rho', \kappa)$ 
7:    $\mathbf{w} := \mathbf{A}\mathbf{y}$  ▷  $\mathbf{w} := \text{NTT}^{-1}(\hat{\mathbf{A}} \cdot \text{NTT}(\mathbf{y}))$ 
8:    $\mathbf{w}_1 := \text{HighBits}_q(\mathbf{w}, 2\gamma_2)$ 
9:    $\tilde{c} \in \{0, 1\}^{256} := H(\mu \parallel \mathbf{w}_1)$ 
10:   $c \in B_\tau := \text{SampleInBall}(\tilde{c})$  ▷ Store  $c$  in NTT representation as  $\hat{c} = \text{NTT}(c)$ 
11:   $\mathbf{z} := \mathbf{y} + c\mathbf{s}_1$  ▷ Compute  $c\mathbf{s}_1$  as  $\text{NTT}^{-1}(\hat{c} \cdot \hat{\mathbf{s}}_1)$ 
12:   $\mathbf{r}_0 := \text{LowBits}_q(\mathbf{w} - c\mathbf{s}_2, 2\gamma_2)$  ▷ Compute  $c\mathbf{s}_2$  as  $\text{NTT}^{-1}(\hat{c} \cdot \hat{\mathbf{s}}_2)$ 
13:  if  $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$  or  $\|\mathbf{r}_0\|_\infty \geq \gamma_2 - \beta$  then
14:     $(\mathbf{z}, \mathbf{h}) := \perp$ 
15:  else
16:     $\mathbf{h} := \text{MakeHint}_q(-c\mathbf{t}_0, \mathbf{w} - c\mathbf{s}_2 + c\mathbf{t}_0, 2\gamma_2)$  ▷ Compute  $c\mathbf{t}_0$  as  $\text{NTT}^{-1}(\hat{c} \cdot \hat{\mathbf{t}}_0)$ 
17:    if  $\|c\mathbf{t}_0\|_\infty \geq \gamma_2$  or the # of 1's in  $\mathbf{h}$  is greater than  $\omega$  then
18:       $(\mathbf{z}, \mathbf{h}) := \perp$ 
19:     $\kappa := \kappa + \ell$ 
20: return  $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$ 

```

기법 2) 구현 코드

- 기법 2 w 압축
 - w 패킹 방법 및 패킹 값에 대한 덧셈 방법

```

/*****
 * Name:      PQCLEAN_MLDSA44_CLEAN_MEM_OPT_polyw_pack
 * ...
 * Arguments: - uint8_t buf[3*256]: output compressed buffer
 * - poly *w: pointer to input polynomial
 *****/
void PQCLEAN_MLDSA44_CLEAN_MEM_OPT_polyw_pack(uint8_t buf[3 * 256], poly *w) {
    // 1. 모듈러 연산 수행 (계수를 q보다 작게 만들)
    PQCLEAN_MLDSA44_CLEAN_MEM_OPT_poly_reduce(w);

    // 2. 음수일 경우 q를 더해 양수로 만들 (0 ~ q-1 범위 보장)
    PQCLEAN_MLDSA44_CLEAN_MEM_OPT_poly_caddq(w);

    unsigned int i;
    for (i = 0; i < N; i++) {
        // 3. 24비트(3바이트) 패킹 과정
        // 하위 8비트 저장
        buf[i * 3 + 0] = w->coeffs[i];

        // 중간 8비트 저장 (8비트 우측 시프트)
        buf[i * 3 + 1] = w->coeffs[i] >> 8;

        // 상위 8비트 저장 (16비트 우측 시프트)
        buf[i * 3 + 2] = w->coeffs[i] >> 16;

        // *참고: w->coeffs[i] >> 24 (최상위 바이트)는 버립니다.
        // 앞선 reduce/caddq 덕분에 상위 바이트는 어차피 0이기 때문입니다.
    }
}

```

```

static void polyw_add_idx(uint8_t buf[3 * 256], int32_t a, size_t i) {
    int32_t coeff;

    // 1. 압축 해제 (Unpack 24-bit to 32-bit)
    coeff = buf[i * 3 + 0];
    coeff |= (int32_t)buf[i * 3 + 1] << 8;
    coeff |= (int32_t)buf[i * 3 + 2] << 16;

    // 2. 값 덧셈
    coeff += a;

    // 3. 모듈러 연산 (freeze는 reduce와 유사하게 값을 q 범위로 맞춤)
    coeff = PQCLEAN_MLDSA44_CLEAN_MEM_OPT_freeze(coeff);

    // 4. 다시 압축 저장 (Repack to 24-bit)
    buf[i * 3 + 0] = coeff;
    buf[i * 3 + 1] = coeff >> 8;
    buf[i * 3 + 2] = coeff >> 16;
}

```

기법 3) 챌린지 c 곱셈과 sparse 활용

- 서명 과정에서의 곱셈 연산 단축
 - 서명 생성 식 : $z = y + c * s_1$ (s_1 : 비밀키, y : 일회용 랜덤 값(마스킹))
- **챌린지 c ?** 서명의 보안성과 무결성을 보장하는 역할
 - 서명자가 비밀키를 알고 있다는 것을 증명하기 위해 풀어야 하는 무작위 문제
 - c는 256개의 계수를 갖지만 대부분이 0이고 아주 적은 수(39~60개)만 ± 1 인 희소(sparse) 다항식
 - 역할 1) 서명하려는 메시지 M과 서명자가 생성한 랜덤값인 w를 섞어서 Hash한 결과
 - 역할 2) 비밀키에 곱해지는 계수 역할
 - 역할 3) 스쿨북 곱셈 최적화에 사용됨
(ex) $c * s_1$ 여기서 0인 부분 생략가능하게 함)

기법 3) 챌린지 c 곱셈과 sparse 활용

- NTT 단점
 - 기존 NTT는 c , t_0 를 모두 NTT 변환하여 메모리에 올려야함
 - ➔ 메모리 과다 사용함
 - ➔ t_0 는 값이 커서 작은 소수($q'=257$)를 쓰는 Alternative NTT를 못 쓰기 때문
- 해결방법
 - 스쿨북 곱셈으로 수정해서 연산
 - 스쿨북 곱셈을 사용함으로써 NTT 변환 없이 필요한 계수만 그때그때 Streaming 연산
 - 챌린지 c 의 희소성 활용
 - c 는 256개의 계수를 갖지만 대부분이 0이고 아주 적은 수(39~60개)만 ± 1 인 희소(sparse) 다항식
 - 0 인 부분 연산 생략
- 효과
 - ± 1 곱셈이기 때문에 곱셈 연산을 단순 덧셈/뺄셈으로 대체할 수 있음
 - 거대한 버퍼 대신에 0인 아닌 계수의 위치(인덱스)만 저장해서 메모리 절약
 - NTT 대비 1.37배 감소

기법 3) 구현 코드

```

/*****
 * Name:      PQCLEAN_MLDSA44_CLEAN_MEM_OPT_poly_challenge_compress
 *
 * Description: Challenge 다항식 c를 압축하는 함수 (논문 기법 1)
 * Challenge 다항식은 대부분의 계수가 0이고,
 * TAU(ML-DSA-44의 경우 39개)개의 계수만 +1 또는 -1 값을 가짐.
 * 따라서 전체 다항식(1024 bytes)을 저장하는 대신,
 * 0이 아닌 계수의 '위치(index)'와 '부호(sign)'만 저장하여 68 bytes로 줄임.
 *
 * Arguments:  - uint8_t c[68]: 압축된 데이터를 저장할 출력 배열 (68 바이트)
 *              - const poly *cp: 입력 Challenge 다항식 (256개 계수)
 *****/
void PQCLEAN_MLDSA44_CLEAN_MEM_OPT_poly_challenge_compress(uint8_t c[68], const poly *cp) {
    unsigned int i, pos;
    uint64_t signs; // 0이 아닌 계수들의 부호를 비트 단위로 저장할 변수
    uint64_t mask;  // 비트 마스크

    /* c 배열 초기화 */
    for (i = 0; i < 68; i++) {
        c[i] = 0;
    }

    signs = 0;
    mask = 1;
    pos = 0;

    /* 다항식의 모든 계수(N=256)를 순회하며 0이 아닌 값 찾기 */
    for (i = 0; i < N; ++i) {
        if (cp->coeffs[i] != 0) {
            // 1. 0이 아닌 계수의 위치(인덱스 i)를 저장 (0~60 바이트 사용)
            c[pos++] = i;

            // 2. 계수의 부호가 -1인 경우 signs 변수의 해당 비트를 1로 설정
            if (cp->coeffs[i] == -1) {
                signs |= mask;
            }
            // 계수가 +1인 경우 해당 비트는 0으로 유지됨

            mask <<= 1; // 다음 비트로 마스크 이동
        }
    }

    /* 저장된 부호 정보(64비트)를 배열의 마지막 8바이트(60~67번 인덱스)에 저장 */
    for (i = 0; i < 8; ++i) {
        c[60 + i] = (unsigned char)(signs >> 8 * i);
    }
}

```

```

/*****
void PQCLEAN_MLDSA44_CLEAN_MEM_OPT_poly_schoolbook(poly *c, const uint8_t ccomp[68]) {
    unsigned i, j, idx;
    uint64_t signs = 0;

    /* 결과 다항식 초기화 */
    for (i = 0; i < N; i++) {
        c->coeffs[i] = 0;
    }

    /* 압축된 데이터의 뒤쪽 8바이트에서 부호 정보 복원 */
    for (i = 0; i < 8; i++) {
        signs |= ((uint64_t)ccomp[60 + i]) << (8 * i);
    }

    /* 0이 아닌 계수들(TAU 개수만큼)에 대해서만 반복 수행 */
    for (idx = 0; idx < TAU; idx++) {
        i = ccomp[idx]; // 0이 아닌 계수의 위치(차수)

        /* Case 1: c[i]가 +1인 경우 (signs 비트가 0) */
        if (!(signs & 1)) {
            // 다항식 곱셈: 결과 += t0 * x^i

            // 1. 차수가 N을 넘지 않는 부분 (단순 덧셈)
            for (j = 0; i + j < N; j++) {
                // c[i+j] += t0[j]
                c->coeffs[i + j] += polyt0_unpack_idx(t0, j);
            }

            // 2. 차수가 N을 넘어가서 모듈러 x^N + 1에 의해 부호가 반대로 돌아오는 부분 (Negacyclic Property)
            // x^N = -1 이므로, x^(N+k) = -x^k 가 됨 (뺄셈 수행)
            for (j = N - i; j < N; j++) {
                // c[i+j-N] -= t0[j]
                c->coeffs[i + j - N] -= polyt0_unpack_idx(t0, j);
            }
        }

        /* Case 2: c[i]가 -1인 경우 (signs 비트가 1) */
        else {
            // 다항식 곱셈: 결과 -= t0 * x^i (위와 부호 반대)

            // 1. 차수가 N을 넘지 않는 부분 (단순 뺄셈)
            for (j = 0; i + j < N; j++) {
                c->coeffs[i + j] -= polyt0_unpack_idx(t0, j);
            }

            // 2. 차수가 N을 넘어가는 부분 (Negacyclic Property)
            // -1 * -1 = +1 이 되므로 덧셈 수행
            for (j = N - i; j < N; j++) {
                c->coeffs[i + j - N] += polyt0_unpack_idx(t0, j);
            }
        }
    }

    signs >>= 1; // 다음 계수의 부호 확인을 위해 쉬프트
}

```

기법 4) Alternative NTT 곱셈 최적화

- Alternative NTT
 - 기존 q 값은 $q = 8,380,417$ 으로 너무 큼
 - $c * s$ 의 결과값은 작은 값이기 때문에 q' 을 써도 결과가 동일
 - Dilithium 2 : 257 / Dilithium 3: 769 / Dilithium 5 : 257
 - 16비트 변수인 65,536 안에 해당되는 가장 작은 소수
 - NTT 연산이 가능한 $k * 256 + 1$ 형태 만족
 - q' 사용을 통해 데이터를 32-bit 대신 16-bit 변수 저장 가능
 - 결과적으로, 메모리 사용량 50% 절감($1.0\text{KiB} = 0.5\text{KiB} \langle c * s_1 \rangle + 0.5\text{KiB} \langle c * s_2 \rangle$)
- 크로네커 치환(Kronecker Substitution) : 이론적 대체 가능한 옵션
 - 다항식 곱셈을 '엄청 큰 정수 곱셈'으로 변경
 - $a_2x^2 + a_1x^1 + a_0 \rightarrow a_2(2^8)^2 + a_1(2^8)^1 + a_0$
 - 다항식이 2,000 비트가 넘는 하나의 Big integer이 됨
 - 결과적으로 Alternative NTT 방식 채택함

사용 기법 정리

연산 대상	기존 방식 (Standard)	대체된 방식 (This work)	이유 및 원리
$c * s1, c * s2$	Standard NTT ($q=8,380,417$)	Alternative NTT ($q' = 257$)	• s의 값이 작아서 257 같은 작은 소수로도 계산 가능함. • 32비트 대신 16비트 연산 사용 ->메모리 절반 절약.
$c * t0$	Standard NTT	Sparse Schoolbook	• $t0$는 값이 커서 257 소수를 못 씀. • 대신 c가 0이 많은 희소(Sparse) 다항식임을 이용. • 0인 부분은 계산 안 하고 건너뛰는 Schoolbook 방식이 더 효율적.

변수 할당

- 메모리 병목 현상이 여러 서브 루틴 간에 공유됨
- w 벡터 원소를 한 번에 하나씩만 저장하는 방식
 - 하지만 이 방법을 쓰면 나중에 w 가 다시 필요해서 적용 안함
 - 처음부터 다시 계산해야한다는 의미
 - $A*y$, NTT 연산등에서 이 서명 생성이 가장 오래걸리는데 이걸 2번 해야 하기 때문
 - 서명 생성 속도가 2배 느려지기 때문

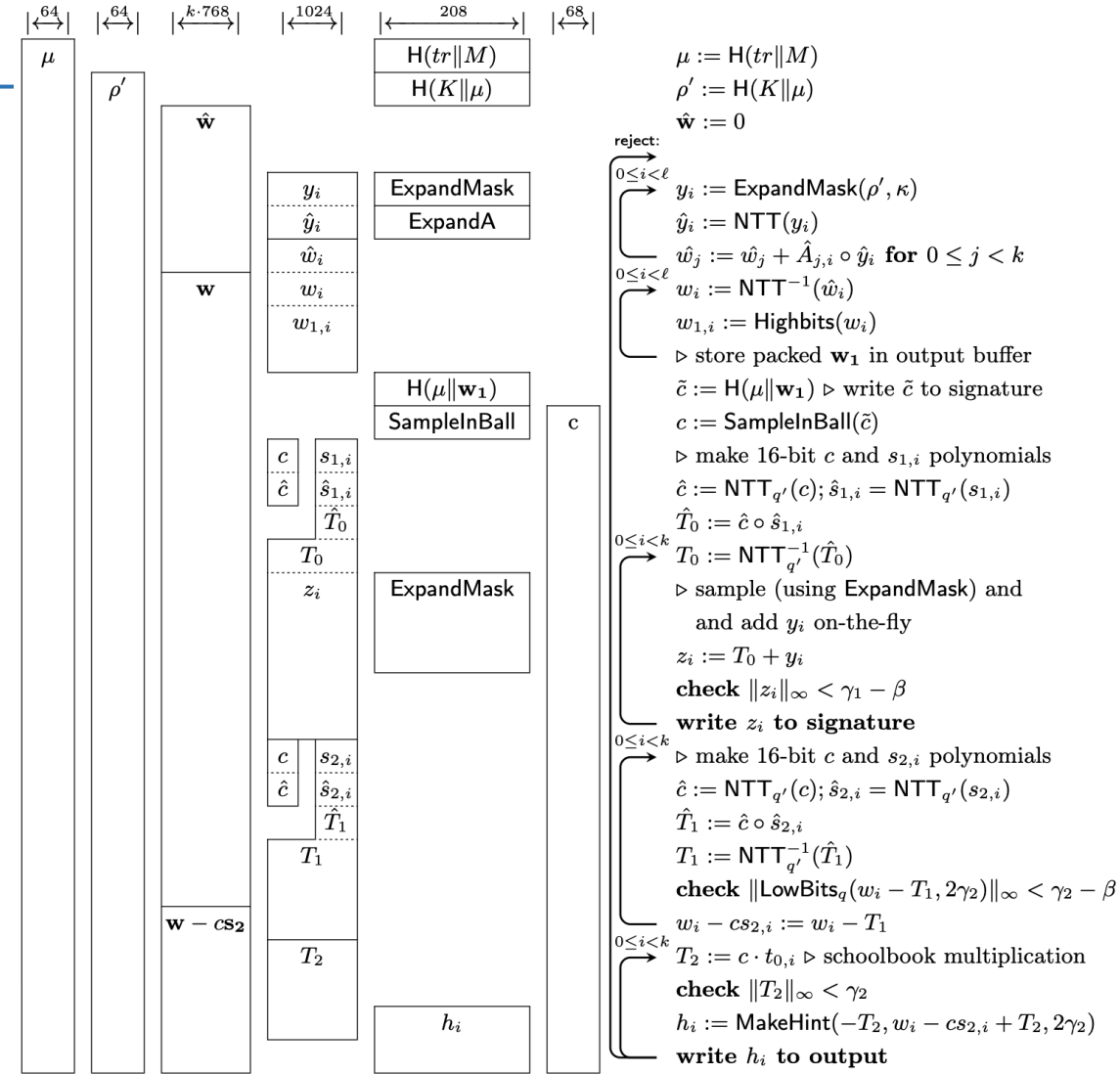


Fig. 1. Memory allocation of the Dilithium signature generation algorithm. Horizontal direction shows the memory slots that are used. Vertical direction shows the progression in time. The boxes indicate the lifetimes of the variables used in the algorithm. Dotted barriers denote that a variable is *renamed*, i.e., it is modified in-place. Arrows in the algorithm indicate loops that iterate over some range, except for the loop annotated by **reject**:, which indicates which code is repeated when a signature in the **Sign** algorithm is aborted. All temporary values are denoted by a T .

키 생성 알고리즘 최적화

- 서명 알고리즘 기법 재사용(스트리밍 기법 사용)
 - $A*s_1$ 곱셈 계산할때 다항식용 별도 메모리 슬롯 제거
 - 스케줄링을 통해 KeyGen 메모리 사용량을 서명 생성 알고리즘보다 줄임

- 키 생성알고리즘은 챌린지 c 와 같은 변수가 없기 때문에 벡터 t 의 원소들이 동시에 메모리에 상주할 필요가 없음
 - **스트리밍 방식으로** 처리
 - $t = A*s_1 + s_2$ 계산 시 원소들의 수명이 겹치지 않도록
 - Dilithium 2 : 3.0KiB 절약 / Dilithium 3: 4.5KiB 절약/ Dilithum 5 :6.0 KiB 절약

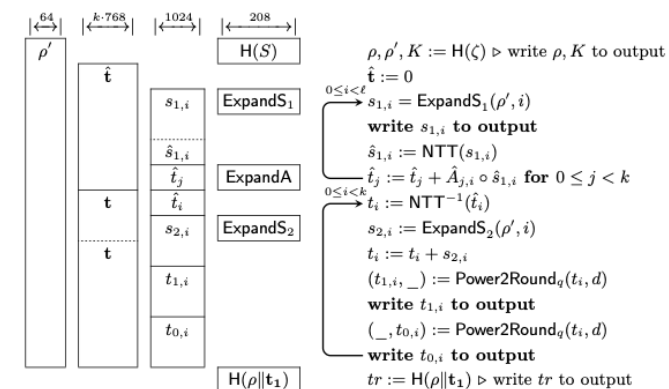


Fig. 2. Memory allocation of the Dilithium key generation algorithm. Horizontal direction shows the memory slots that are used. Vertical direction shows the progression in time. The boxes indicate the lifetimes of the variables used in the algorithm. Dotted barriers denote that a variable is *renamed*, i.e., it is modified in-place. Arrows in the algorithm indicate loops that iterate over some range.

서명 검증 알고리즘

- 메모리 구조
 - 다항식 슬롯 2개만 있으면 충분(1.0KiB+768Byte)
 - 기존) 행렬 A 나 벡터 w를 계산하기 위해 모든 다항식을 메모리에 한꺼번에 올려두고 연산
 - 하지만, Streaming(순차적)으로 수행하여 동시에 딱 2개의 다항식만 메모리에 위치하면 됨
 - 슬롯 1 (누적값 저장용, 1.0 KiB)
 - 슬롯2 (임시 연산용, 768byte)
 - (SHA-3 state(208Byte)*2) + 압축된 c (68Byte)
- 약 2.3 KiB의 고정된 메모리만 필요함
 - 다항식 슬롯 2개+필수 데이터(SHA3 state 등)
 - **서명 검증 알고리즘은 딜리시움 레벨과 상관없이 메모리 사용량 일정함**

성능 결과

최종 메모리 사용량

함수	PQClean	논문 (This work)	mem_opt	절약률
키 생성	36.2 KB	4.9 KB	3.2 KB	91%
서명 생성	49.4 KB	5.0 KB	6.0 KB	88%
서명 검증	34.0 KB	2.7 KB	2.0 KB	94%

측정 방식 차이

항목	논문 (This work)	mem_opt (본 구현)
측정 도구	pqm4 Stack Canary	GCC <code>-fstack-usage</code>
측정 방식	런타임 동적 측정	컴파일 타임 정적 분석
서브루틴	자동 포함 (Peak 측정)	수동으로 합산
플랫폼	ARM Cortex-M4	macOS, Apple Silicon

논문의 측정 원리 (Stack Canary Method):

- 실행 전 스택을 더미 값(0xDEADBEEF)으로 채움
- 알고리즘 실행
- 덮어쓰워진 영역을 스캔하여 Peak 사용량 측정

본 구현의 측정 방식:

- GCC `-fstack-usage` : 각 함수의 스택 프레임 크기만 측정
- 서브루틴 호출 시 추가 스택은 수동으로 합산 (예: 서명 생성 = 4,656 + 1,504)
- 실제 런타임 Peak와 차이가 있을 수 있음

측정 환경: macOS, Apple Silicon, `-Os` 최적화

성능 비교

함수	PQClean	mem_opt	비율
키 생성	167 μ s	204 μ s	1.2x
서명 생성	451 μ s	743 μ s	1.6x
서명 검증	82 μ s	141 μ s	1.7x

측정 환경: macOS, Apple Silicon, `-O3` 최적화, 100회 평균

Table 2. Memory usage and cycle counts for Dilithium in kibibytes (KiB) and kilocycles (kcc). K, S, and V correspond to the signing primitives Key-Gen, Sign, and Verify respectively. All cycle counts were averaged over 10 000 iterations.

	variant	Dilithium-2		Dilithium-3		Dilithium-5		
		KiB	kcc	KiB	kcc	KiB	kcc	
		total ≤ 8		total ≤ 8		total ≤ 8		
with asm	[7] (pqm4)	K	37.1	1 602	59.6	2 835	95.7	4 835
		S	47.9	4 219	72.3	6 742	112.3	8 960
		V	35.2	1 579	56.6	2 700	90.8	4 718
	[1]	K	37.1	1 598	59.6	2 830	95.7	4 828
		S	47.9	4 083	67.4	6 624	113.3	8 726
		V	35.2	1 572	56.6	2 692	90.8	4 707
C only	PQClean	K	37.4	2 025	59.4	3 504	— ^a	— ^a
		S	50.7	8 034	77.7	12 987	— ^a	— ^a
		V	35.4	2 223	56.4	3 666	— ^a	— ^a
	This work	K	4.9 ✓	2 927	6.4 ✓	5 112	7.9 ✓	8 609
		S	5.0 ✓	18 470	6.5 ✓	36 303	8.1	44 332
		V	2.7 ✓	4 036	2.7 ✓	7 249	2.7 ✓	12 616

^a Implementation disabled because the device does not have enough RAM to support it.

Table 3. Code sizes of the implementations from Table 2 expressed in bytes. Opt-level denotes the optimization level that was used. Contribution of Keccak and AES code is excluded from all implementations.

implementation	opt-level	Dilithium-2	Dilithium-3	Dilithium-5
[7] (pqm4)	-O3	10 564	10 092	— ^a
[1]	-O3	18 448	19 916	18 262
[7] (pqm4)	-Os	9 700	9 276	— ^a
[1]	-Os	17 408	19 012	17 234
PQClean	-Os	6 986	6 534	— ^b
This work	-Os	10 091^c	10 091^c	10 091^c

^a Not reported by pqm4.

^b Implementation disabled because the device does not have enough RAM to support it.

^c Implementation includes support for all Dilithium variants.

Q & A